

Smytten E-Commerce Database Management System

CHAPTER – 9

PAYMENT DETAILS MANAGEMENT

Objectives:

- Create Review and Rating tables.
- Store customer feedback and ratings.
- Retrieve product review details.
- Calculate average product ratings using aggregate functions.
- Identify highly rated products.

1. Database Design

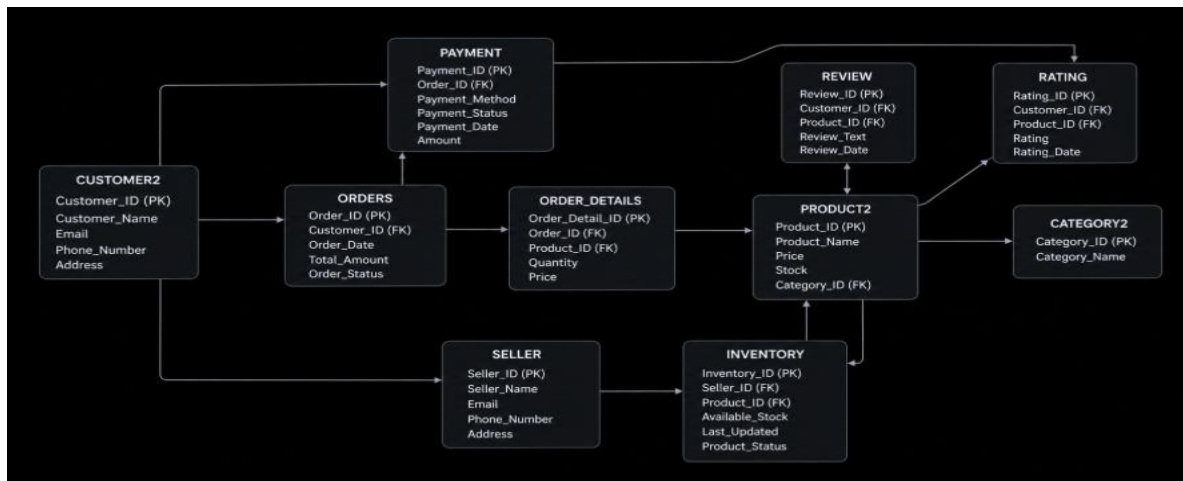
1.1 Review Table

Attribute	Data Type	Constraint
Review_ID	NUMBER	Primary Key
Customer_ID	NUMBER	Foreign Key, NOT NULL
Product_ID	NUMBER	Foreign Key, NOT NULL
Review_Text	VARCHAR2(500)	NOT NULL
Review_Date	DATE	NOT NULL

1.2 Rating Table:

Attribute	Data Type	Constraint
Rating_ID	NUMBER	Primary Key
Customer_ID	NUMBER	Foreign Key, NOT NULL
Product_ID	NUMBER	Foreign Key, NOT NULL
Rating	NUMBER(1)	NOT NULL, CHECK (1–5)
Rating_Date	DATE	NOT NULL

2. Table Relationships:



3. Table Creation

3.1 Create Review Table

```
SQL> CREATE TABLE REVIEW (
2  REVIEW_ID NUMBER PRIMARY KEY,
3  CUSTOMER_ID NUMBER NOT NULL,
4  PRODUCT_ID NUMBER NOT NULL,
5  REVIEW_TEXT VARCHAR2(500),
6  REVIEW_DATE DATE DEFAULT SYSDATE,
7  CONSTRAINT FK_REVIEW_CUSTOMER FOREIGN KEY (CUSTOMER_ID) REFERENCES CUSTOMER2(CUSTOMER_ID),
8  CONSTRAINT FK_REVIEW_PRODUCT FOREIGN KEY (PRODUCT_ID) REFERENCES PRODUCT2(PRODUCT_ID)
9 );
```

Table created.

3.2 Create Rating Table

```
SQL> CREATE TABLE RATING (
2  RATING_ID NUMBER PRIMARY KEY,
3  CUSTOMER_ID NUMBER NOT NULL,
4  PRODUCT_ID NUMBER NOT NULL,
5  RATING NUMBER(1) NOT NULL CHECK (RATING BETWEEN 1 AND 5),
6  RATING_DATE DATE DEFAULT SYSDATE,
7  CONSTRAINT FK_RATING_CUSTOMER FOREIGN KEY (CUSTOMER_ID) REFERENCES CUSTOMER2(CUSTOMER_ID),
8  CONSTRAINT FK_RATING_PRODUCT FOREIGN KEY (PRODUCT_ID) REFERENCES PRODUCT2(PRODUCT_ID)
9 );
```

Table created.

4. Review Data Insertion

```
SQL> INSERT INTO REVIEW VALUES (501,101,101,'Very good product and quality is excellent',TO_DATE('20-09-2026','DD-MM-YYYY'))
1 row created.
SQL> INSERT INTO REVIEW VALUES (502,102,102,'Good product and worth the price',TO_DATE('21-09-2026','DD-MM-YYYY'));
1 row created.
SQL> INSERT INTO REVIEW VALUES (503,103,103,'Product quality is average',TO_DATE('22-09-2026','DD-MM-YYYY'));
1 row created.
SQL> INSERT INTO REVIEW VALUES (504,104,101,'Really useful product. Loved it',TO_DATE('23-09-2026','DD-MM-YYYY'));
1 row created.
SQL> INSERT INTO REVIEW VALUES (505,105,104,'Amazing product and fast delivery',TO_DATE('24-09-2026','DD-MM-YYYY'));
1 row created.
SQL> INSERT INTO REVIEW VALUES (506,106,102,'Product is good',TO_DATE('25-09-2026','DD-MM-YYYY'));
1 row created.
SQL> INSERT INTO REVIEW VALUES (507,107,105,'Excellent quality and packaging',TO_DATE('26-09-2026','DD-MM-YYYY'));
1 row created.
SQL> INSERT INTO REVIEW VALUES (508,108,103,'Not fully satisfied with the product',TO_DATE('27-09-2026','DD-MM-YYYY'));
1 row created.
```

5. Rating Data Insertion

```
SQL> INSERT INTO RATING VALUES (601,101,101,5,TO_DATE('20-09-2026','DD-MM-YYYY'));
1 row created.

SQL> INSERT INTO RATING VALUES (602,102,102,4,TO_DATE('21-09-2026','DD-MM-YYYY'));
1 row created.

SQL> INSERT INTO RATING VALUES (603,103,103,3,TO_DATE('22-09-2026','DD-MM-YYYY'));
1 row created.

SQL> INSERT INTO RATING VALUES (604,104,101,5,TO_DATE('23-09-2026','DD-MM-YYYY'));
1 row created.

SQL> INSERT INTO RATING VALUES (605,105,104,5,TO_DATE('24-09-2026','DD-MM-YYYY'));
1 row created.

SQL> INSERT INTO RATING VALUES (606,106,102,4,TO_DATE('25-09-2026','DD-MM-YYYY'));
1 row created.

SQL> INSERT INTO RATING VALUES (607,107,105,5,TO_DATE('26-09-2026','DD-MM-YYYY'));
1 row created.
```

6. Retrieve Product Review Details

```
SQL> SELECT REVIEW_ID,CUSTOMER_ID,PRODUCT_ID,REVIEW_TEXT,REVIEW_DATE
2 FROM REVIEW
3 ORDER BY REVIEW_DATE;
```

```
REVIEW_ID CUSTOMER_ID PRODUCT_ID
-----
REVIEW_TEXT
-----
REVIEW_DA
-----
          501          101          101
Very good product and quality is excellent
20-SEP-26
```

```
          502          102          102
Good product and worth the price
21-SEP-26
```

```
REVIEW_ID CUSTOMER_ID PRODUCT_ID
-----
REVIEW_TEXT
-----
REVIEW_DA
-----
          503          103          103
Product quality is average
22-SEP-26
```

7. Calculate Average Product Ratings

This uses the AVG() aggregate function.

```
SQL> SELECT PRODUCT_ID,  
2 COUNT(RATING) AS TOTAL_RATINGS,  
3 ROUND(AVG(RATING),2) AS AVERAGE_RATING  
4 FROM RATING  
5 GROUP BY PRODUCT_ID  
6 ORDER BY AVERAGE_RATING DESC;
```

PRODUCT_ID	TOTAL_RATINGS	AVERAGE_RATING
101	2	5
104	1	5
105	1	5
110	1	5
109	1	4
102	2	4
103	2	2.5

7 rows selected.

8. Identify Highly Rated Products

Products with an average rating of 4 or above can be displayed using HAVING.

```
SQL> SELECT PRODUCT_ID,  
2 ROUND(AVG(RATING),2) AS AVERAGE_RATING  
3 FROM RATING  
4 GROUP BY PRODUCT_ID  
5 HAVING AVG(RATING) >= 4  
6 ORDER BY AVERAGE_RATING DESC;
```

PRODUCT_ID	AVERAGE_RATING
101	5
104	5
105	5
110	5
109	4
102	4

6 rows selected.

9. Product Rating Report

A combined report can be generated using the Rating and Product tables.

```
SQL> SELECT P.PRODUCT_ID,  
2 P.PRODUCT_NAME,  
3 COUNT(R.RATING) AS TOTAL_RATINGS,  
4 ROUND(AVG(R.RATING),2) AS AVERAGE_RATING  
5 FROM PRODUCT2 P  
6 JOIN RATING R  
7 ON P.PRODUCT_ID=R.PRODUCT_ID  
8 GROUP BY P.PRODUCT_ID,P.PRODUCT_NAME  
9 ORDER BY AVERAGE_RATING DESC;
```

PRODUCT_ID	PRODUCT_NAME	TOTAL_RATINGS	AVERAGE_RATING
101	Face Wash	2	5
104	Mascara	1	5

```
SQL> SELECT R.REVIEW_ID,  
2 R.CUSTOMER_ID,  
3 R.PRODUCT_ID,  
4 R.REVIEW_TEXT,  
5 RT.RATING,  
6 R.REVIEW_DATE  
7 FROM REVIEW R  
8 JOIN RATING RT  
9 ON R.PRODUCT_ID=RT.PRODUCT_ID  
10 AND R.CUSTOMER_ID=RT.CUSTOMER_ID  
11 ORDER BY R.PRODUCT_ID;
```

```
REVIEW_ID CUSTOMER_ID PRODUCT_ID  
-----
```

```
REVIEW_TEXT  
-----
```

```
      RATING REVIEW_DA  
-----  
      501      101      101  
Very good product and quality is excellent  
5 20-SEP-26  
  
      504      104      101  
Really useful product. Loved it  
5 23-SEP-26
```